

Pipeline FASTq file to BAM & VCF

I. Download the data from CCG

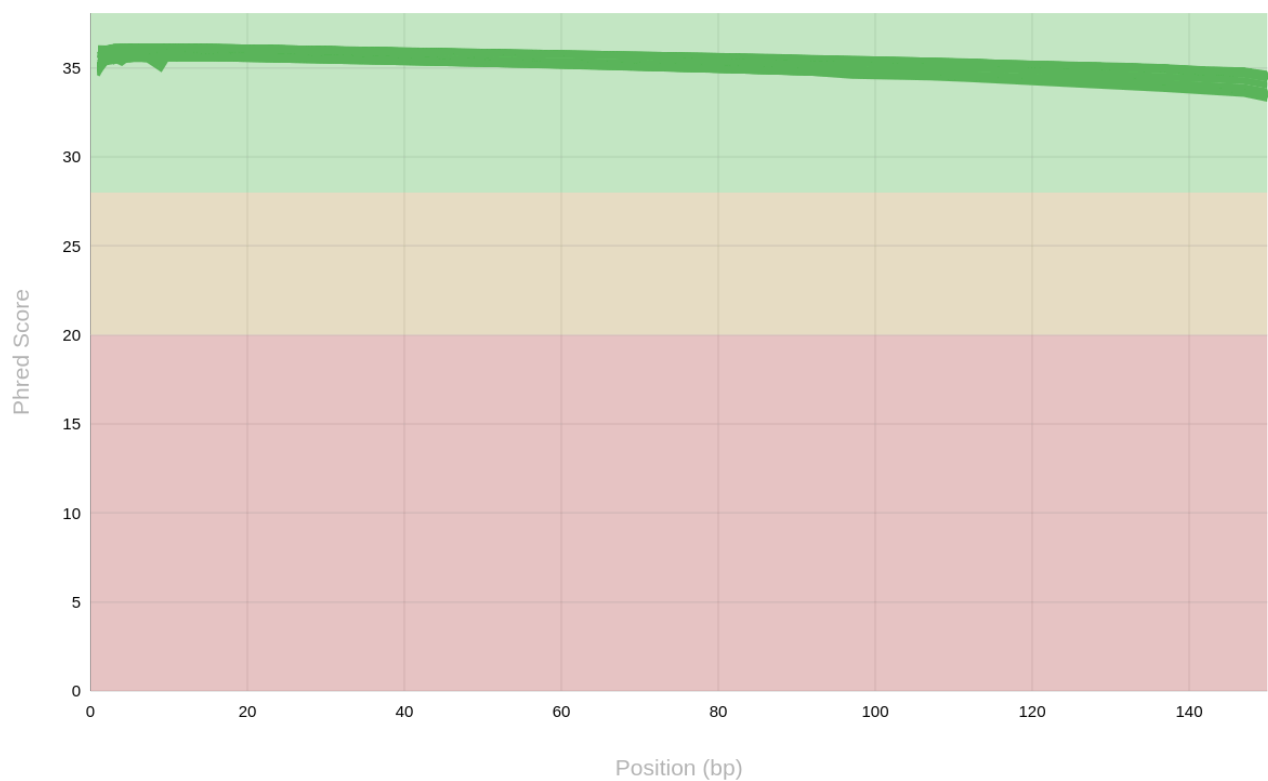
1. Connect to Rames and download the data directly into the ag-demeaux folder

```
sftp NGS_25D382_jflorete_DX467@delivery.ccg.uni-koeln.de
NGS_25D382_jflorete_DX467@delivery.ccg.uni-koeln.de's password: <your_delivery_pwd>
ls 01102025 #name of the folder by the ccg
get -r 01102025
quit
```

1. Checking the fastqc multi report for the quality

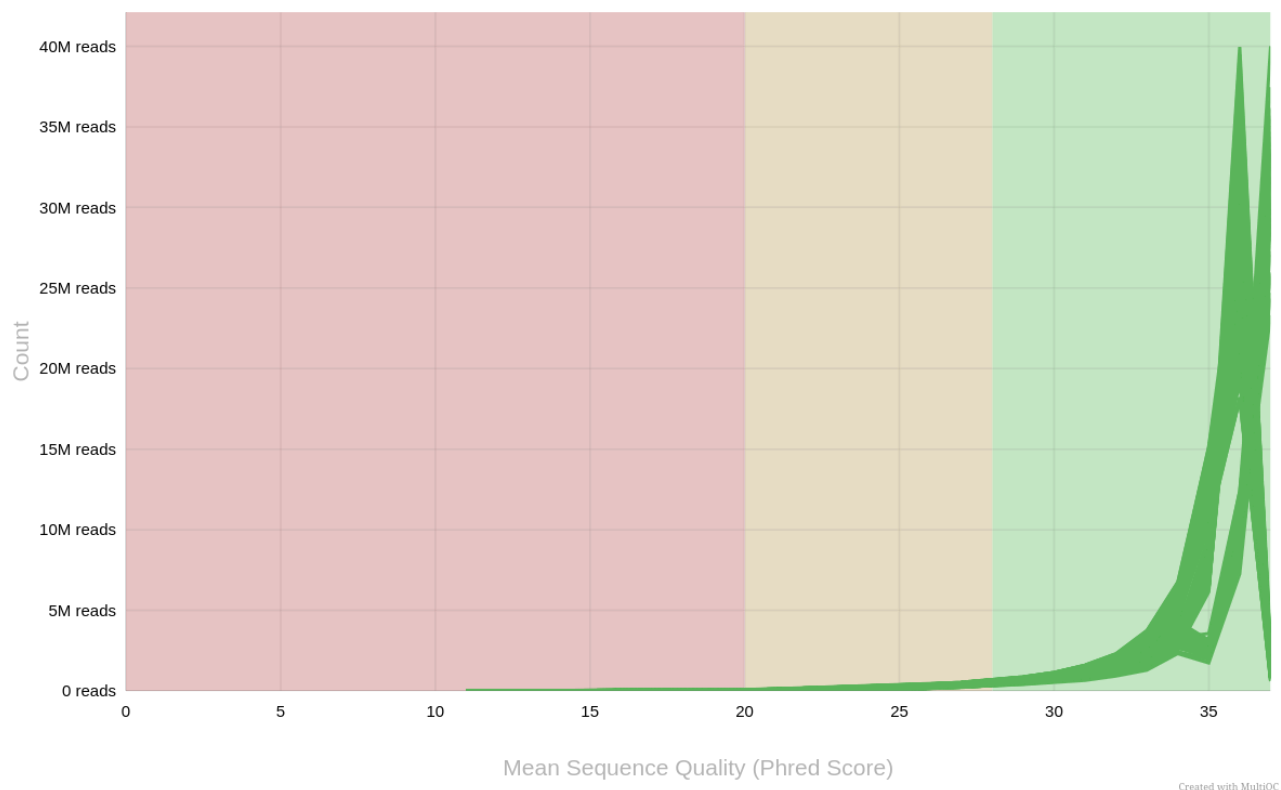
Good quality no need to filter more the fastq files !

FastQC: Mean Quality Scores

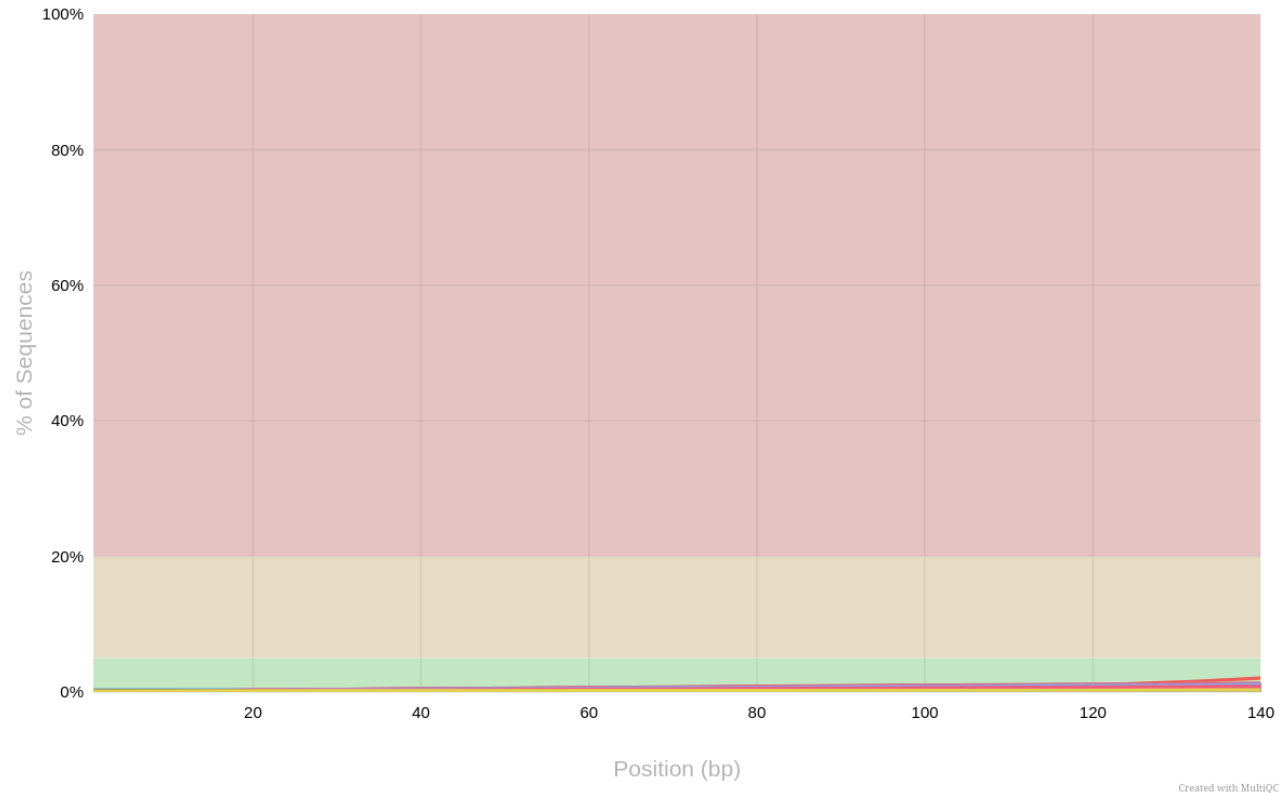


Created with MultiQC

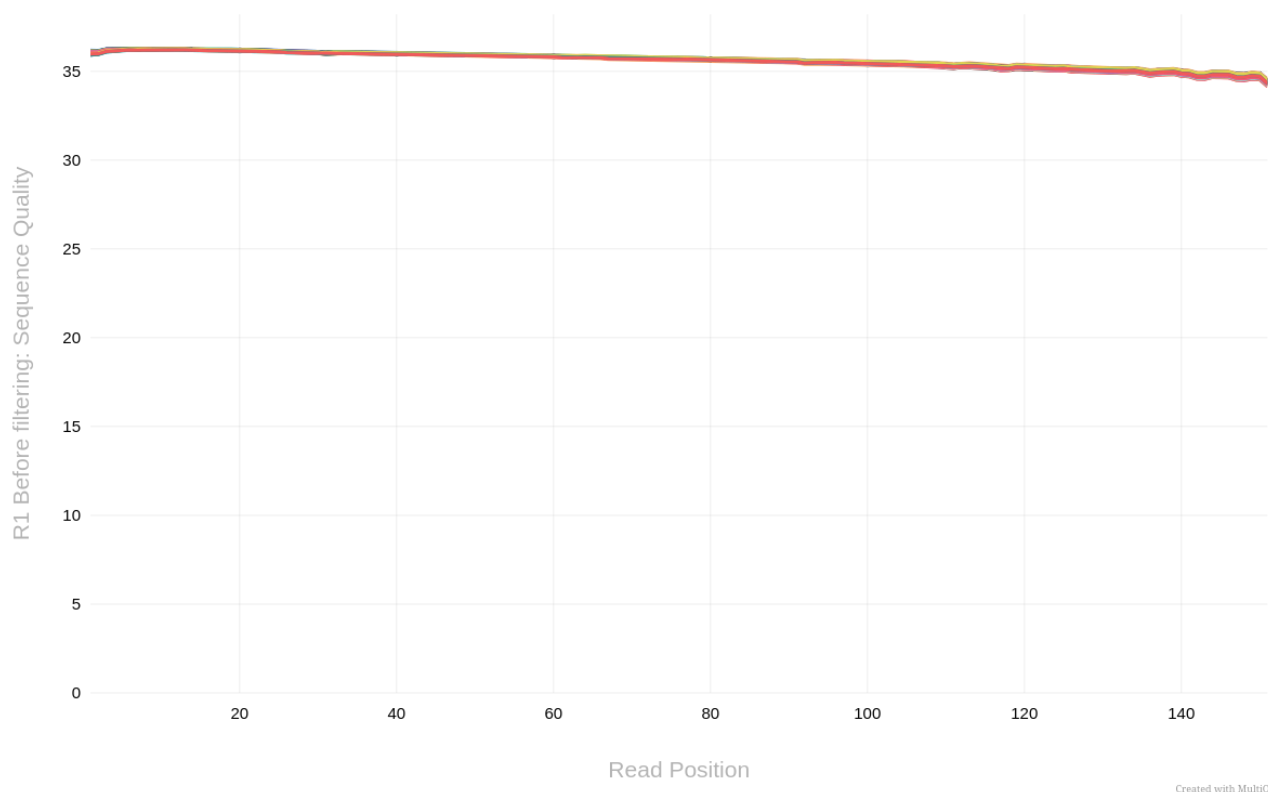
FastQC: Per Sequence Quality Scores



FastQC: Adapter Content



Fastp: Sequence Quality



Supplementary information from the CCG

When you run adapter Trimming, use the following sequences, more information:

<https://support-docs.illumina.com/SHARE/AdapterSequences/Content/SHARE/FrontPages/AdapterSeq.htm>

Read1:

- AGATCGGAAGAGCACACGTCTGAACTCCAGTCA
- AGATCGGAAGAGCACACGTCTGAAC
- TGAATTCTCGGGTGCCAAGG
- AGATCGGAAGAGCACACGTCT
- CTGTCTCTTATACACATCT
- AGATGTGTATAAGAGACAG

Read2:

- AGATCGGAAGAGCGTCGTGTAGGGAAAGAGTGT
- AGATCGGAAGAGCGTCGTGTAGGGA
- TGAATTCTCGGGTGCCAAGG
- AGATCGGAAGAGCACACGTCT
- CTGTCTCTTATACACATCT
- AGATGTGTATAAGAGACAG

UMI for PCR duplicate

The multiplex kit, that was used to sequence your data, contains UMIs. Those Unique Molecular Identifiers can be used to filter out PCR duplicates, but it is not mandatory to use the data for that purpose. You do not have to use that information, it is optional data. If you expected R1 and R2, then you will find that data in R1 and R3. R1 and R3 are the normal paired end reads, R2 is the UMI which some people use in downstream analysis.

II. Mapping FASTQ to reference

1. Reference genome: TAIR10

Can be found in:

```
/projects/ag-demeaux/OLD/tali/arabidopsis_lyrata/Arabidopsis_lyrata/a.thaliana_genome
```

Copy paste it in my folder `Ref_genome`

Create an index/database of your reference:

```
module load bio/bwa-mem2/2.2.1-intel-compilers-2023.1.0
bwa-mem2 index ~/QTL_analysis/reference_fasta/GCF_000001735.4_TAIR10.1_genomic.fna`
```

Reference genome pwd `/projects/ag-demeaux/jfloret/Ath_Cologne_2025/Ref_genome`

1.b create a pseudo genome based on one of the two parents MIL2

- first check if the name of the chromosome in vcf file of the parent and the FASTA of the reference are the same

```
bcftools view -h MIL2_KAD1.recode.vcf.gz | grep "^##contig"
grep ">" /projects/ag-demeaux/jfloret/Ath_Cologne_2025/Ref_genome/TAIR10_chr_all.fas
```

- if they are different, modify the name in one of the file, here the vcf file
Create a txt with the corresponding name

```
1   Chr1
2   Chr2
3   Chr3
4   Chr4
5   Chr5
chloroplast   ChrC
mitochondria  ChrM
```

```
bcftools annotate --rename-chrs chr_map.txt MIL2_KAD1.recode.vcf.gz -Oz -o
MIL2_KAD1.recode_renamed.vcf.gz
```

- I add a mismatch SNP between my ref Col0 genome and the vcf file, so I just removed the problematic SNP

```
bcftools +fixref MIL2_KAD1.recode_renamed.vcf.gz \
-- -f /projects/ag-demeaux/jfloret/Ath_Cologne_2025/Ref_genome/TAIR10_chr_all.fas \
> fixref_report.txt
```

It is telling me I have 155 mismatch

```
# NS, Number of sites:
NS   total      2684739
NS   ref match   2534973 100.0%
NS   ref mismatch 155 0.0%
```

- Delete the wrong position

```
bcftools view -T ^bad_site.txt \
-Oz -o MIL2_KAD1.recode_renamed.clean.vcf.gz \
MIL2_KAD1.recode_renamed.vcf.gz
```

Check if only one position have been remove

```
bcftools stats MIL2_KAD1.recode_renamed.vcf.gz | grep "number of records"
bcftools stats MIL2_KAD1.recode_renamed.clean.vcf.gz | grep "number of records"
```

- then index the new vcf file

```
bcftools index MIL2_KAD1.recode_renamed.clean.vcf.gz
```

- create the pseudo reference genome

```
bcftools consensus MIL2_KAD1.recode_renamed.clean.vcf.gz --sample A006200066_116048_S97_L004 --
fasta-ref /projects/ag-demeaux/jfloret/Ath_Cologne_2025/Ref_genome/TAIR10_chr_all.fas >
MIL2_Cologne_line_reference_genome.fa
```

```
samtools dict MIL2_Cologne_line_reference_genome.fa -o MIL2_Cologne_line_reference_genome.dict
```

2. Mapping script

The Scripts are from Tahir Ali adapted by me

1. install the UMI tool

```
git clone [https://github.com/CGATOxford/UMI-tools.git](https://github.com/CGATOxford/UMI-
tools.git)
cd UMI-tools/
module unload compiler/GCCcore
module unload tools/bzip2
module load lang/Python
python setup.py install --user
cd ~/.local/bin/
ls
```

```
f2py fonttools futurize numpy-config pasteurize pyftmerge pyftsubset ttx umi_tools
```

```
./umi_tools
# set path variable in the .bashrc
echo 'export PATH=$HOME/.local/bin:$PATH' >> ~/.bashrc
source ~/.bashrc
umi_tools
```

2. Run the script: [bwa_mapping_umi.sh](#)

```
#!/bin/bash
#####
# bwa_mapping_umi.sh
# Mapping + UMI extraction + deduplication
# Script from Tahir Ali, modified by Justine Floret 11.2025
#####

#SBATCH --time=48:00:00
#SBATCH --cpus-per-task=6
#SBATCH --mem=20gb
#SBATCH --account=ag-demeaux
#SBATCH --error=/projects/ag-demeaux/jfloret/Ath_Cologne_2025/mapping_MIL2/mapping_%a.err
#SBATCH --output=/projects/ag-demeaux/jfloret/Ath_Cologne_2025/mapping_MIL2/mapping_%a.out
#SBATCH --array=1-79

# Purge any previously loaded modules
module purge
```

```

# Load necessary modules
module load compiler/GCCcore/12.3.0
module load compiler/intel-compilers/2023.1.0
module load bio/bwa-mem2/2.2.1-intel-compilers-2023.1.0
module load bio/SAMtools/1.18-GCC-12.3.0

# Define directories and reference genome
REFERENCE="/projects/ag-demeaux/jfloret/Ath_Cologne_2025/Ref_genome/TAIR10_chr_all.fas"
#REFERENCE="/projects/ag-demeaux/jfloret/Mil2-KAD1_lines/MIL2_Cologne_line_reference_genome.fa"
INPUT_DIR="/projects/ag-demeaux/jfloret/Ath_Cologne_2025/Fasta"
OUTPUT_DIR="/projects/ag-demeaux/jfloret/Ath_Cologne_2025/mapping_MIL2"

# Define the path to the sample-to-pool mapping file
SAMPLE_POOL_MAPPING_FILE="/projects/ag-demeaux/jfloret/Ath_Cologne_2025/Sample_Names.tab"

# Create output directory if it doesn't exist
mkdir -p "$OUTPUT_DIR"

# Ensure mapping file exists
if [ ! -f "$SAMPLE_POOL_MAPPING_FILE" ]; then
    echo "Error: Sample-to-pool mapping file not found! Exiting."
    exit 1
fi

# Create a mapping between sample name and pool using an associative array
declare -A sample_pool_map

# Read the sample-pool mapping file, skipping header
while IFS=$'\t' read -r sample pool; do
    if [[ "$sample" == "Sample_ID" || -z "$sample" ]]; then
        continue
    fi
    sample_pool_map["$sample"]="$pool"
done < "$SAMPLE_POOL_MAPPING_FILE"

# Get array of all R1 files
R1_FILES=("${INPUT_DIR}/*_R1_*.fastq.gz")

# Check if array task ID is valid
if [ ${SLURM_ARRAY_TASK_ID} -gt ${#R1_FILES[@]} ]; then
    echo "Error: SLURM_ARRAY_TASK_ID ${SLURM_ARRAY_TASK_ID} exceeds number of files
${#R1_FILES[@]}"
    exit 1
fi

# Select file for this array task (arrays are 1-indexed in SLURM, bash arrays are 0-indexed)
R1="${R1_FILES[${SLURM_ARRAY_TASK_ID} - 1]}"

echo "==> Array task ${SLURM_ARRAY_TASK_ID} processing file: $R1"

# Ensure input file exists
if [ ! -f "$R1" ]; then
    echo "ERROR: Forward read file $R1 not found."
    exit 1
fi

# Define paired reads
R2="${R1/_R1/_R2_}"
R3="${R1/_R1/_R3_}"

if [ ! -f "$R2" ]; then
    echo "ERROR: Reverse read file $R2 not found for $R1."
    exit 1
fi

if [ ! -f "$R3" ]; then
    echo "ERROR: R3 file not found for $R1."

```

```

    exit 1
fi

# Extract read header
header=$(zcat "$R1" | head -n 1)
id=$(echo "$header" | cut -f 1-4 -d ":" | sed 's/@/\/' | sed 's/:/_/g')

# Extract numeric sample ID from filename
SAMPLE=$(basename "$R1" | cut -d '_' -f2)

# Lookup pool info
lb="${sample_pool_map[$SAMPLE]}"
if [ -z "$lb" ]; then
    echo "No pool info for sample $SAMPLE. Defaulting to 'unknown'."
    lb="unknown"
fi

# Define output files
bam_umi="${OUTPUT_DIR}/${SAMPLE}_umi.bam"
bam_dedup="${OUTPUT_DIR}/${SAMPLE}_dedup.bam"
R1_extracted="${OUTPUT_DIR}/${SAMPLE}_R1_extracted.fq.gz"
R2_extracted="${OUTPUT_DIR}/${SAMPLE}_R2_extracted.fq.gz"
R3_extracted="${OUTPUT_DIR}/${SAMPLE}_R3_extracted.fq.gz"

echo "==> Processing sample: ${SAMPLE}"

# Step 1: Extract UMIs from R2 and transfer to both R1 and R3
# R2 is the "primary" input (contains UMI), R1 is "secondary" output
umi_tools extract \
    --bc-pattern=NNNNNNNNNNNN \
    --stdin "$R2" \
    --read2-in "$R1" \
    --stdout >(gzip > "$R2_extracted") \
    --read2-out >(gzip > "$R1_extracted") \
    --log "${OUTPUT_DIR}/${SAMPLE}_umi_extract_R1.log"

# Extract same UMI from R2 and transfer to R3
umi_tools extract \
    --bc-pattern=NNNNNNNNNNNN \
    --stdin "$R2" \
    --read2-in "$R3" \
    --stdout >(gzip > "$R2_extracted") \
    --read2-out >(gzip > "$R3_extracted") \
    --log "${OUTPUT_DIR}/${SAMPLE}_umi_extract_R3.log"

# Step 2: Map extracted reads
bwa-mem2 mem -t "${SLURM_CPUS_PER_TASK}" \
    -R "@RG\tID:${id}_PE\tSM:$SAMPLE\tLB:$lb\tPL:ILLUMINA" \
    "$REFERENCE" "$R1_extracted" "$R3_extracted" \
    | samtools view -bS - \
    | samtools sort -o "$bam_umi"

# Index BAM before deduplication
samtools index "$bam_umi"

# Step 3: Deduplicate using UMI-tools
umi_tools dedup \
    --stdin "$bam_umi" \
    --stdout "$bam_dedup" \
    --paired \
    --log "${OUTPUT_DIR}/${SAMPLE}_umi_dedup.log"

samtools index "$bam_dedup"

# Step 4: Clean up intermediate files
echo "Cleaning up intermediate files for ${SAMPLE}..."
rm -f "$R1_extracted" "$R2_extracted" "$R3_extracted"

```

```
echo "==> Finished processing sample: ${SAMPLE}"
echo "May the force be with you - Task ${SLURM_ARRAY_TASK_ID} complete!"
```

3. BAM qualimap report on the quality

1. Install Qualimap on the server

```
# Download Qualimap
wget https://bitbucket.org/kokonech/qualimap/downloads/qualimap_v2.3.zip

# Unzip
unzip qualimap_v2.3.zip
cd qualimap_v2.3

# Unload any conflicting modules (optional, depending on your cluster)
module purge
module load lang/R/4.4.1-gfbb-2023b

# Install R dependencies
Rscript scripts/installDependencies.R

# If there is an issue, install your own library
mkdir -p ~/R/x86_64-pc-linux-gnu-library/4.4
export R_LIBS_USER=~/R/x86_64-pc-linux-gnu-library/4.4
echo $R_LIBS_USER #/home/jfloret/R/x86_64-pc-linux-gnu-library/4.4

# BiocManager installation package
R
if (!requireNamespace("BiocManager", quietly = TRUE))
  install.packages("BiocManager", repos = "https://cran.r-project.org")

BiocManager::install(c("NOISeq"), ask = FALSE)
q()
```

- Edit the Qualimap launch script:

```
nano qualimap
```

- Press Ctrl+W and search for MaxPermSize
- Find the line: `java_options="-Xms32m -Xmx$JAVA_MEM_SIZE -XX:MaxPermSize=1024m"`
- Change it to: `java_options="-Xms32m -Xmx$JAVA_MEM_SIZE"` - Save (Ctrl+O, Enter) and exit (Ctrl+X)
- Set Up R Environment for Plotting with Qualimap

```
mkdir -p ~/R_libs
echo '.libPaths("~/R/x86_64-pc-linux-gnu-library/4.4")' >> ~/.Rprofile
```

In R:

```
.libPaths()
```

- add qualimap to bashrc

```
echo 'export PATH=/projects/ag-demeaux/jfloret/tools/qualimap_v2.3:$PATH' >> ~/.bashrc
source ~/.bashrc
```

2. Run qualimap per sample: [qualimap.sh](#)


```
#!/bin/bash -l

#####
# qualimap.sh
# Mapping quality
# Script from Tahir Ali, modified by Justine Floret 11.2025
#####

#SBATCH --job-name=bamqc
#SBATCH --time=24:00:00
#SBATCH --cpus-per-task=4
#SBATCH --mem=16gb
#SBATCH --account=ag-demeaux
#SBATCH --error=/projects/ag-demeaux/jfloret/Ath_Cologne_2025/mapping_MIL2/qualimap/logs/bamqc%A_%a.err
#SBATCH --output=/projects/ag-demeaux/jfloret/Ath_Cologne_2025/mapping_MIL2/qualimap/logs/bamqc%A_%a.out
#SBATCH -D /scratch/jfloret
#SBATCH --array=1-79          # Adjust based on the number of BAM files

# Load modules if needed
# module load lang/R/4.4.1-gfbf-2023b # only if you need Java, optional if qualimap is in PATH

# Define paths
BAM_DIR="/projects/ag-demeaux/jfloret/Ath_Cologne_2025/mapping"
QC_DIR="/projects/ag-demeaux/jfloret/Ath_Cologne_2025/mapping/qualimap"
mkdir -p "${QC_DIR}"

#-----
# Build list of BAM files only for IDs 325553-325607
#-----
#mapfile -t BAM_FILES < < (
#   for ID in $(seq 325553 325607); do
#       FILE="${BAM_DIR}/${ID}_dedup.bam"
#       [[ -f "$FILE" ]] && echo "$FILE"
#   done
#)

# Get list of BAM files
mapfile -t BAM_FILES < < (find "${BAM_DIR}" -maxdepth 1 -name "*dedup.bam" | sort)

# Validate SLURM_ARRAY_TASK_ID
if [[ -z "${SLURM_ARRAY_TASK_ID}" || "${SLURM_ARRAY_TASK_ID}" -le 0 || "${SLURM_ARRAY_TASK_ID}" -gt "${#BAM_FILES[@]}" ]]; then
    echo "ERROR: SLURM_ARRAY_TASK_ID (${SLURM_ARRAY_TASK_ID}) is out of range! Exiting..."
    exit 1
fi

# Select BAM file based on SLURM array index
BAM_FILE="${BAM_FILES[${SLURM_ARRAY_TASK_ID} - 1]}"
BAM_BASENAME=$(basename "${BAM_FILE}" .bam)
OUT_DIR="${QC_DIR}/${BAM_BASENAME}_qc"

echo "=====
Running Qualimap on: ${BAM_FILE}
Output Directory: ${QC_DIR}/${BAM_BASENAME}_qc
=====

# Create output directory
mkdir -p "${QC_DIR}/${BAM_BASENAME}_qc"

# Run Qualimap
qualimap bamqc -bam "${BAM_FILE}" -outdir "${QC_DIR}/${BAM_BASENAME}_qc" -outformat HTML --java-mem-size=8G &> "${OUT_DIR}/qualimap.log"
```

```
echo "Finished BAM QC for: ${BAM_FILE}"
```

For the parents

```
qualimap bamqc -bam KAD1.bam -outdir qualimap/KAD1 -outformat HTML --java-mem-size=8G &>
qualimap/qualimap.log
qualimap bamqc -bam MIL2.bam -outdir qualimap/MIL2 -outformat HTML --java-mem-size=8G &>
qualimap/qualimap.log
```

3. MultiQC report for all the sample: [multiqc.sh](#)

```
#!/bin/bash -l

#####
# multiqc.sh
# Multiqc report of the mapping quality
# Script from Tahir Ali, modified by Justine Floret 11.2025
#####

#SBATCH --job-name=multiqc
#SBATCH --time=1:00:00
#SBATCH --cpus-per-task=4
#SBATCH --mem=16gb
#SBATCH --account=ag-demeaux
#SBATCH --error=/projects/ag-demeaux/jfloret/Ath_Cologne_2025/mapping/qualimap/logs/multiqc.err
#SBATCH --output=/projects/ag-demeaux/jfloret/Ath_Cologne_2025/mapping/qualimap/logs/multiqc.out

# Load MultiQC module
module purge
module load bio/MultiQC/1.22.3-foss-2023b

# Define paths
QC_DIR="/projects/ag-demeaux/jfloret/Ath_Cologne_2025/mapping/qualimap"
MULTIQC_OUT="${QC_DIR}/multiqc_report"
mkdir -p "${MULTIQC_OUT}"

echo "=====
echo "Running MultiQC on Qualimap BAM QC results..."
echo "Output Directory: ${MULTIQC_OUT}"
echo "=====

# Run MultiQC
multiqc "${QC_DIR}" -o "${MULTIQC_OUT}"
echo "MultiQC report generated in: ${MULTIQC_OUT}"
```

4. Check the coverage depth along the genome

- Install jvarkit

```
module purge
module load lang/Java/17.0.6
git clone --recurse-submodules "https://github.com/lindenb/jvarkit.git"
cd jvarkit
./gradlew jvarkit
```

- Create the dictionary file

```
module load bio/picard/3.0.0-Java-17
java -jar $EBROOTPICARD/picard.jar CreateSequenceDictionary \
  R=TAIR10_chr_all.fas \
  O=TAIR10_chr_all.dict
```

- Making sure the chromosome name in the fasta dict. and bam are the same

```
grep ">" TAIR10_chr_all.fas | head
samtools idxstats 325554_dedup.bam | head
```

- Run [coverage_depth.sh](#)

```
#!/bin/bash -l

#####
# generate_coverage.sh
# Estimate Coverage_depth of each sample before moving to SNP calling
# Script to generate coverage plots and calculate average coverage for a list of BAM files
# Script from Tahir Ali, modified by Justine Floret
#####

#SBATCH --nodes=1
#SBATCH --ntasks-per-node=8
#SBATCH --mem=20gb
#SBATCH --time=5:00:00
#SBATCH --account=ag-demeaux
#SBATCH --error=/projects/ag-demeaux/jfloret/Ath_Cologne_2025/mapping/CoverageDepth/coverage_svg-
%x-%j.err
#SBATCH --output=/projects/ag-
demeaux/jfloret/Ath_Cologne_2025/mapping_/CoverageDepth/coverage_svg-%x-%j.out

module purge
module load lang/Java/17.0.6
module load bio/SAMtools/1.18-GCC-12.3.0

#-----
# Directories and input files
#-----

output_dir="/projects/ag-demeaux/jfloret/Ath_Cologne_2025/mapping/CoverageDepth"
input_dir="/projects/ag-demeaux/jfloret/Ath_Cologne_2025/mapping"
ref_genome="/projects/ag-demeaux/jfloret/Ath_Cologne_2025/Ref_genome/TAIR10_chr_all.fas"
set -euo pipefail

#-----
# Automatically detect all *_dedup.bam files
#-----
mapfile -t sample_bams < <(find "${input_dir}" -maxdepth 1 -type f -name "*_dedup.bam" | sort)

# Check that BAMs were found
if [[ ${#sample_bams[@]} -eq 0 ]]; then
    echo "ERROR: No *_dedup.bam files found in ${input_dir}"
    exit 1
fi

#-----
# Main loop
#-----
for bam_path in "${sample_bams[@]"; do
    bam_file=$(basename "${bam_path}")
    sample_name="${bam_file%_dedup.bam}"
    sample_dir="${output_dir}/${sample_name}"
    output_svg="${sample_dir}/${sample_name}.svg"

    mkdir -p "${sample_dir}"

    echo "=====
    echo "Processing sample: ${sample_name}"
    echo "BAM file: ${bam_file}"
```

```

echo "Output directory: ${sample_dir}"
echo "=====

#-----
# Generate coverage plot using JVARKit
#-----
# Adjust --include-contig-regex to your chromosome naming (e.g. Chr1, Chr2, ...)
java -jar /projects/ag-demeaux/jfloret/tools/jvarkit/dist/jvarkit.jar wgscoverageplotter \
    --dimension 1500x500 \
    -C -1 \
    --clip \
    -R "${ref_genome}" \
    "${input_dir}/${bam_file}" \
    --include-contig-regex "Chr.*" \
    --percentile median > "${output_svg}"

echo "Coverage plot for ${sample_name} saved as ${output_svg}"

#-----
# Calculate average coverage using samtools depth + awk
#-----
samtools depth -a "${input_dir}/${bam_file}" > "${sample_dir}/coverage.txt"
average_coverage=$(awk 'sum += $3 END {if (NR>0) print sum/NR; else print 0}'
"${sample_dir}/coverage.txt")
echo "Average Coverage for ${sample_name}: ${average_coverage}"

#-----
# Calculate coverage using samtools coverage
#-----
samtools coverage "${input_dir}/${bam_file}" > "${sample_dir}/coverage_samtools.txt"
echo "Coverage calculation for ${sample_name} completed."
echo
done

echo "All coverage plots and calculations completed successfully."

```

For the parents

```

java -jar /projects/ag-demeaux/jfloret/tools/jvarkit/dist/jvarkit.jar wgscoverageplotter \
    --dimension 1500x500 \
    -C -1 \
    --clip \
    -R /projects/ag-demeaux/jfloret/Ath_Cologne_2025/Ref_genome/TAIR10_chr_all.fas \
    ../KAD1.bam \
    --include-contig-regex "Chr.*" \
    --percentile median > KAD1.svg

echo "Coverage plot for ${sample_name} saved as ${output_svg}"

```

- creating a pdf with all the svg file:

```

# move every svg file in one folder
mv 325*/*.svg svg_file/

# with inkscape merge into one pdf, with one svg file per page
cd svg_file/
mkdir -p pdf_tmp && for f in *.svg; do inkscape "$f" --export-type=pdf --export-
filename="pdf_tmp/$(basename "${f%.svg}").pdf"; done && pdfunite pdf_tmp/*.pdf merged_output.pdf
&& rm -r pdf_tmp

```

III. SNP calling

The SNP calling included the bam file of the two parents MIL2 - KAD1 based on the mapping on Col0 reference genome.

1. Call the SNP

run the script: [snpcalling.sh](#)

```
#!/bin/bash -l

#####
# SNP calling script
# Script from Tahir Ali, modified by Justine Floret
# 24/11/2025
#####

#SBATCH --nodes=1
#SBATCH --time=72:00:00
#SBATCH --cpus-per-task=8
#SBATCH --mem=40gb
#SBATCH --account=ag-demeaux
#SBATCH --error=/projects/ag-demeaux/jfloret/Ath_Cologne_2025/SNP_calling/vcf_refCol0/bcftools_%j.err
#SBATCH --output=/projects/ag-demeaux/jfloret/Ath_Cologne_2025/SNP_calling/vcf_refCol0/bcftools_%j.out

module purge
module load bio/BCFtools/1.19-GCC-13.2.0
ulimit -n 2003

# Define directories
bam_dir="/projects/ag-demeaux/jfloret/Ath_Cologne_2025/mapping_MIL2/"
output_vcf="/projects/ag-demeaux/jfloret/Ath_Cologne_2025/SNP_calling/vcf_refCol0"

# Ensure output directory exists
mkdir -p ${output_vcf}

# Run bcftools mpileup on all BAM files in the directory
bcftools mpileup \
  -a AD,DP,SP \
  -q 40 \
  -Q 30 \
  -Ou \
  -f /projects/ag-demeaux/jfloret/Ath_Cologne_2025/Ref_genome/TAIR10_chr_all.fas \
  -b /projects/ag-demeaux/jfloret/Ath_Cologne_2025/mapping/bam_list.txt | bcftools call \
  -m -v \
  -a GQ,GP \
  -Oz \
  -o /projects/ag-demeaux/jfloret/Ath_Cologne_2025/SNP_calling/vcf_refCol0/ath_Cologne_refCol0_B1_2025.vcf.gz
```

2. Soft filtering

[snpcalling_SoftFilter](#)

```
#!/bin/bash
set -euo pipefail

#####
# SNP filtering script
# Script from Tahir Ali, modified by Justine Floret and Margarita Takou
# 12.2025
#####
```

```

#SBATCH --nodes=1
#SBATCH --time=24:00:00
#SBATCH --cpus-per-task=8
#SBATCH --mem=20gb
#SBATCH --account=ag-demeaux
#SBATCH --error=/projects/ag-demeaux/jfloret/Ath_Cologne_2025/SNP_calling/vcf_refCol0/snpfiltering_%j.err
#SBATCH --output=/projects/ag-demeaux/jfloret/Ath_Cologne_2025/SNP_calling/vcf_refCol0/snpfiltering_%J.out

module purge
module load bio/BCFtools/1.19-GCC-13.2.0

# set working directory
cd /projects/ag-demeaux/jfloret/Ath_Cologne_2025/SNP_calling/vcf_refCol0 # adjust if needed

# INPUT
VCF="ath_Cologne_refCol0_B1_2025.vcf.gz"

echo "=== STARTING VCF FILTERING PIPELINE ==="

#####
# 1. Remove indels, other variants → keep only biallelic SNPs
#####
echo "[1] Filtering SNPs only..."

# -g: filter SNPs within _INT_ base pairs of an indel or other other variant type
# -M2 -v snps to only view biallelic SNPs

bcftools filter -Ou -g 5:indel,other $VCF | \
  bcftools view -Oz -M2 -V indels,other,mnps \
  -o ath_Cologne_refCol0_B1_2025_noindel.vcf.gz

bcftools index ath_Cologne_refCol0_B1_2025_noindel.vcf.gz
echo "SNP count:"
bcftools view -H ath_Cologne_refCol0_B1_2025_noindel.vcf.gz | wc -l

#####
# 2. MQ, QUAL, and Strand Bias (SP) filters
#####
echo "[3] Applying MQ, QUAL, SP filters..."

# MQ: minimum mapping quality
bcftools filter -Oz -e 'MQ<40' ath_Cologne_refCol0_B1_2025_noindel.vcf.gz \
  -o ath_Cologne_refCol0_B1_2025_noindel_MQ40.vcf.gz
bcftools index ath_Cologne_refCol0_B1_2025_noindel_MQ40.vcf.gz

# QUAL: site call quality
bcftools filter -Oz -e 'QUAL<30' ath_Cologne_refCol0_B1_2025_noindel_MQ40.vcf.gz \
  -o ath_Cologne_refCol0_B1_2025_noindel_MQ40_QUAL30.vcf.gz
bcftools index ath_Cologne_refCol0_B1_2025_noindel_MQ40_QUAL30.vcf.gz

# SP Phred-scaled strand bias P-value
SP_THRESH=50

bcftools filter -Oz -e "SP>$SP_THRESH" ath_Cologne_refCol0_B1_2025_noindel_MQ40_QUAL30.vcf.gz \
  -o ath_Cologne_refCol0_B1_2025_noindel_MQ40_QUAL30_SP50.vcf.gz
bcftools index ath_Cologne_refCol0_B1_2025_noindel_MQ40_QUAL30_SP50.vcf.gz

echo "Remaining sites after SP filter:"
bcftools view -H ath_Cologne_refCol0_B1_2025_noindel_MQ40_QUAL30_SP50.vcf.gz | wc -l

#####
# 4. Soft filtering (GQ)
#####
echo "[7] Soft filtering: GQ>=25 ..."

```

```
bcftools filter -Ou -S . -e 'FMT/GQ<25'
ath_Cologne_refCol0_B1_2025_noindel_MQ40_QUAL30_SP50.vcf.gz \
-o ath_Cologne_refCol0_B1_2025_noindel_MQ40_QUAL30_SP50_GQ25.vcf.gz

bcftools index ath_Cologne_refCol0_B1_2025_noindel_MQ40_QUAL30_SP50_GQ25.vcf.gz

echo "Sites after soft filtering:"
bcftools view -H ath_Cologne_refCol0_B1_2025_noindel_MQ40_QUAL30_SP50_GQ25.vcf.gz | wc -l

echo "=== FILTERING COMPLETE ==="
```

3. Rename the pool in the vcf file

[/projects/ag-demeaux/jfloret/Ath_Cologne_2025/Sample_Names.tab](#)

```
325543 113
325545 159
325549 173
325605 F2_1
325606 F2_2
325607 F2_3
A006200066_116050_S98_L004 Kad1
A006200066_116048_S97_L004 Mil2
```

```
# checking the vcf name
bcftools query -l ath_Cologne_refCol0_B1_2025_noindel_MQ40_QUAL30_SP50_GQ25.vcf.gz | head

bcftools reheader -s /projects/ag-demeaux/jfloret/Ath_Cologne_2025/Sample_Names.tab -o
ath_Cologne_refCol0_B1_2025_noindel_MQ40_QUAL30_SP50_GQ25_ID.vcf.gz
ath_Cologne_refCol0_B1_2025_noindel_MQ40_QUAL30_SP50_GQ25.vcf.gz

bcftools query -l ath_Cologne_refCol0_B1_2025_noindel_MQ40_QUAL30_SP50_GQ25_ID.vcf.gz | head

mv ath_Cologne_refCol0_B1_2025_noindel_MQ40_QUAL30_SP50_GQ25_ID.vcf.gz
ath_Cologne_refCol0_B1_2025_noindel_MQ40_QUAL30_SP50_GQ25.vcf.gz

bcftools index ath_Cologne_refCol0_B1_2025_noindel_MQ40_QUAL30_SP50_GQ25.vcf.gz
```

4. Filter missing GT in Mil2 - Kad1

```
#####
# 1) KEEP ONLY Mil2 and Kad1                                     8
#####

bcftools view \
-S Parent_list.txt \
-Oz -o Mil2_Kad1_refCol0_noindel_MQ40_QUAL30_SP50_GQ25.vcf.gz \
ath_Cologne_refCol0_B1_2025_noindel_MQ40_QUAL30_SP50_GQ25.vcf.gz

bcftools index Mil2_Kad1_refCol0_noindel_MQ40_QUAL30_SP50_GQ25.vcf.gz

#####
# 2) Allelic frequency and genotype of the parents
#####

bcftools view \
-e 'GT="mis"' \
Mil2_Kad1_refCol0_noindel_MQ40_QUAL30_SP50_GQ25.vcf.gz -Ou \
| bcftools query \
-f '%CHROM\t%POS[\t%GT]\n' \
```

```
> Mil2_Kad1_noNA_sites.txt

bcftools view \
  -e 'GT="mis"' \
  Mil2_Kad1_refCol0_noindel_MQ40_QUAL30_SP50_GQ25.vcf.gz -Ou \
  | bcftools query \
  -f '%CHROM\t%POS\t%REF\t%ALT[\t%SAMPLE\t%GT\t%AD\t%DP]\n' \
  > Mil2_Kad1_GT_positions.txt
```

5. Filter the vcf F2 pools

As documented in bcftools issues, there is currently no way to say “apply this per-sample threshold from an external file” in a single bcftools command. The closest fully supported solution is:

- Use per-site functions like `MEAN(FMT/DP)` or `sMEAN(FMT/DP)` for **site-level** filtering.
- Or move the true per-sample $2 \times \text{meanDP}$ logic into a custom script (Python/R) that reads the VCF and your mean file together.

If you confirm you are okay with the site-level approximation (`DP>2×mean_site_DP` instead of `DP>2×mean_sample_DP`), the one-line `+setGT` command above is the robust bcftools implementation.

vcf_F2_allelic_filtering.sh

```
#!/bin/bash
set -euo pipefail

#####
# vcf filtering for analysis script
# Script from Justine Floret
# 12.2025
#####

#SBATCH --nodes=1
#SBATCH --time=24:00:00
#SBATCH --cpus-per-task=8
#SBATCH --mem=10gb
#SBATCH --account=ag-demeaux
#SBATCH --error=/projects/ag-demeaux/jfloret/Ath_Cologne_2025/SNP_calling/vcf_refCol0/vcffiltering_%j.err
#SBATCH --output=/projects/ag-demeaux/jfloret/Ath_Cologne_2025/SNP_calling/vcf_refCol0/vcffiltering_%J.out

module purge
module load bio/BCFtools/1.19-GCC-13.2.0

# INPUT
VCF="ath_Cologne_refCol0_B1_2025_noindel_MQ40_QUAL30_SP50_GQ25.vcf.gz"

cd /projects/ag-demeaux/jfloret/Ath_Cologne_2025/SNP_calling/vcf_refCol0/

echo "=== STARTING VCF FILTERING PIPELINE ==="

#####
# 1. KEEP ONLY F2 POOLS
#####
echo "=== KEEPING ONLY F2 POOLS ==="

bcftools view \
  -S F2_list \
  -Oz -o F2_ath_Cologne_refCol0.vcf.gz \
  "$VCF"
```



```
bcftools index F2_ath_Cologne_refCol0.vcf.gz
```

```
#####  
# 2. KEEP ONLY THE POSITION WITH NO NA IN THE PARENTS  
#####  
echo "=== KEEPING POSITION WITH NI NA IN PARENTS ==="
```

```
bcftools view \  
-T Mil2_Kad1_noNA_sites.txt \  
-Oz -o F2_ath_Cologne_refCol0_noNaGT.vcf.gz \  
F2_ath_Cologne_refCol0.vcf.gz
```

```
bcftools index F2_ath_Cologne_refCol0_noNaGT.vcf.gz
```

```
#####  
# 3. REMOVE MISSING GT "./."  
#####  
echo "=== Remove the missing genotype ./. ==="
```

```
bcftools view \  
-e 'GT="mis"' \  
-Oz -o F2_ath_Cologne_refCol0_noMissingGT.vcf.gz \  
F2_ath_Cologne_refCol0_noNaGT.vcf.gz
```

```
bcftools index F2_ath_Cologne_refCol0_noMissingGT.vcf.gz
```

```
NB_SNP=$(bcftools view -v snps F2_ath_Cologne_refCol0_noMissingGT.vcf.gz | grep -v '^#' | wc -l)  
echo "Number of SNP is: $NB_SNP"
```

```
#####  
# 4. Compute mean DP and number of SNP per chromosome  
#####  
echo "[2] Number of SNP per chromosome..."
```

```
FILE=F2_ath_Cologne_refCol0_noMissingGT.vcf.gz
```

```
bcftools query -f '%CHROM[\t%SAMPLE\t%GT]\n' "$FILE" \  
| awk '  
{  
  chr = $1  
  for (i=2; i<=NF; i+=2) {  
    s = $i  
    gt = $(i+1)  
    if (gt != "./." && gt != ".") {  
      key = chr FS s  
      cnt[key]++  
    }  
  }  
}  
END {  
  for (k in cnt) print k, cnt[k]  
}' OFS='\t' \  
| sort -k1,1 -k2,2 > NB_SNP_F2_per_chr_per_sample.tsv
```

```
echo "[3] Mean DP per Sample..."  
# Rows = sites, columns = samples  
bcftools query -f ' [%DP\t]\n' "$FILE" > F2_DP_matrix.txt
```

```
# Streaming per-sample mean with awk  
awk '  
{  
  if (NR==1) ncol = NF  
  for (i=1; i<=NF; i++) if ($i != ".") { sum[i] += $i; n[i]++ }  
}  
END{  
  for (i=1; i<=ncol; i++) {
```

```

    mean = (n[i]>0 ? sum[i]/n[i] : 0)
    print i, mean
}
}' F2_DP_matrix.txt > F2_DP_mean_per_column.txt

#####
# 5. KEEP ONLY SNPs IN GENE REGIONS
#####
echo "=== KEEPING ONLY SNP IN GENE REGIONS ==="

bcftools view \
-R /projects/ag-demeaux/jfloret/Ath_Cologne_2025/Ref_genome/TAIR10_genes.bed \
-Oz -o F2_ath_Cologne_refCol0_noMissingGT_genic.vcf.gz \
F2_ath_Cologne_refCol0_noMissingGT.vcf.gz

bcftools index F2_ath_Cologne_refCol0_noMissingGT_genic.vcf.gz

bcftools query -f '%CHROM[\t%SAMPLE\t%GT]\n' F2_ath_Cologne_refCol0_noMissingGT_genic.vcf.gz \
| awk '
{
    chr = $1
    for (i=2; i<=NF; i+=2) {
        s = $i
        gt = $(i+1)
        if (gt != "." && gt != ".") {
            key = chr FS s
            cnt[key]++
        }
    }
}
END {
    for (k in cnt) print k, cnt[k]
}' OFS='\t' \
| sort -k1,1 -k2,2 > NB_SNP_F2_per_chr_per_sample_genic.tsv

#####
# 6. DP FILTERING
#####
echo "=== Per sample per SNP filtering DP ==="

# Create a file with the mean DP per sample in the same order as in the vcf file and indexed
paste F2_list F2_DP_mean_per_column.txt \
| awk '{print $1, $3}' OFS='\t' > F2_DP_mean_per_sample.txt
# cols: SAMPLE MEAN_DP

#Add a 1-based index, then convert to 0-based
nl -w1 -s' ' F2_DP_mean_per_sample.txt \
| awk '{idx0=$1-1; sample=$2; mean=$3; print idx0, sample, mean}' OFS='\t' \
> F2_DP_mean_per_sample_idx.txt

cp F2_ath_Cologne_refCol0_noMissingGT_genic.vcf.gz tmp_DPclean.vcf.gz
bcftools index tmp_DPclean.vcf.gz

while read -r IDX SAMPLE MEAN_DP; do
    THR=$(awk -v m="$MEAN_DP" 'BEGIN{printf "%.2f", 2*m}')
    echo "Filtering index $IDX ($SAMPLE) with thresholds 10 <= DP <= $THR"

    bcftools +setGT tmp_DPclean.vcf.gz \
    -Oz -o tmp_DPclean2.vcf.gz \
    -- -t q -n . \
    -i "FMT/DP[$IDX]<10 | FMT/DP[$IDX]>$THR"

    mv tmp_DPclean2.vcf.gz tmp_DPclean.vcf.gz
    bcftools index tmp_DPclean.vcf.gz
done < F2_DP_mean_per_sample_idx.txt

```

```
mv tmp_DPclean.vcf.gz F2_ath_Cologne_refCol0_noMissingGT_genic_DPclean.vcf.gz
bcftools index F2_ath_Cologne_refCol0_noMissingGT_genic_DPclean.vcf.gz
```

```
# Number of SNP
```

```
bcftools query -f '%CHROM[\t%SAMPLE\t%GT]\n'
F2_ath_Cologne_refCol0_noMissingGT_genic_DPclean.vcf.gz \
| awk '
{
    chr = $1
    for (i=2; i<=NF; i+=2) {
        s = $i
        gt = $(i+1)
        if (gt != "./." && gt != ".") {
            key = chr FS s
            cnt[key]++
        }
    }
}
END {
    for (k in cnt) print k, cnt[k]
}' OFS='\t' \
| sort -k1,1 -k2,2 > NB_SNP_F2_per_chr_per_sample_genic_DPclean.tsv
```

```
#####
```

```
# 7. ALLELIC FREQUENCY R FRIENDLY TABLE
```

```
#####
```

```
echo "=== Extracting the allelic frequency in R friendly table ==="
```

```
bcftools query \
-f '%CHROM\t%POS\t%REF\t%ALT[\t%SAMPLE\t%GT\t%AD\t%DP]\n' \
F2_ath_Cologne_refCol0_noMissingGT_genic_DPclean.vcf.gz \
| awk '
{
    chrom=$1; pos=$2; ref=$3; alt=$4;
    # fields from 5 onwards are groups of 4: SAMPLE, GT, AD, DP
    for (i=5; i<=NF; i+=4) {
        sample=$(i); gt=$(i+1); ad=$(i+2); dp=$(i+3);
        altfreq="NA";
        if (ad != "." && dp != "." && dp > 0) {
            split(ad,a,","); # a[1]=REF reads, a[2]=ALT reads (for biallelic)
            altfreq = a[2] / dp;
        }
        print chrom"\t"pos"\t"ref"\t"alt"\t"sample"\t"gt"\t"ad"\t"dp"\t"altfreq;
    }
}
' > F2_ath_Cologne_refCol0_noMissingGT_genic_DPclean_RfriendlyAF.tsv
# Column header: CHROM POS REF ALT SAMPLE GT AD DP ALTFREQ
```

```
#####
```

```
# 8. GENIC SNPs WITH 0.4–0.6 ALLELE FREQ POSITION
```

```
#####
```

```
echo "=== KEEPING POSITION OF SNP BETWEEN 0.4 - 0.6 FREQUENCY ==="
```

```
# Per-pool AF for genic SNPs
```

```
bcftools query \
-f '%CHROM\t%POS\t%REF\t%ALT[\t%SAMPLE\t%GT\t%AD\t%DP]\n' \
F2_ath_Cologne_refCol0_noMissingGT_genic_DPclean.vcf.gz |
awk '
{
    chrom=$1; pos=$2; ref=$3; alt=$4;
    for(i=5;i<=NF;i+=3){
        sample=$(i);
        split($(i+1),ad,",");
        dp=$(i+2);
        if (dp==0 || ad[2]=="." || ad[2]=="") af="NA";
```

```

        else af=ad[2]/dp;
        printf("%s\t%s\t%s\t%s\t%s\t%f\n",chrom,pos,ref,alt,sample,af);
    }
}' > F2_ath_Cologne_refCol0_noMissingGT_genic_DPclean_positions.tsv

# Extracting the position at 0.4 and 0.6 allelic frequency
awk '($6 >= 0.4 && $6 <= 0.6)' F2_ath_Cologne_refCol0_noMissingGT_genic_DPclean_positions.tsv \
| cut -f1-2 \
| sort -k1,1 -k2,2n -u \
> sites_genic_DPclean_noMissingGT_AF0.4_0.6.txt

echo "=== MAY THE FORCE BE WITH YOU ==="

```

6. Filter the vcf F5 test pools

vcf_F5_allelic_filtering.sh

```

#!/bin/bash
set -euo pipefail

#####
# vcf filtering for analysis script
# Script from Justine Floret
# 12.2025
#####

#SBATCH --nodes=1
#SBATCH --time=24:00:00
#SBATCH --cpus-per-task=8
#SBATCH --mem=10gb
#SBATCH --account=ag-demeaux
#SBATCH --error=/projects/ag-demeaux/jfloret/Ath_Cologne_2025/SNP_calling/vcf_refCol0/vcffiltering_%j.err
#SBATCH --output=/projects/ag-demeaux/jfloret/Ath_Cologne_2025/SNP_calling/vcf_refCol0/vcffiltering_%J.out

module purge
module load bio/BCFtools/1.19-GCC-13.2.0

# set working directory
cd /projects/ag-demeaux/jfloret/Ath_Cologne_2025/SNP_calling/vcf_refCol0 # adjust if needed

echo "=== STARTING VCF FILTERING PIPELINE ==="

#####
# 1) KEEP ONLY F5 POOLS
#####

echo "=== KEEPING ONLY F5 POOLS ==="

bcftools view \
-S F5_list.txt \
-Oz -o F5_refCol0_B1_2025_noindel_MQ40_QUAL30_SP50_GQ25.vcf.gz \
ath_Cologne_refCol0_B1_2025_noindel_MQ40_QUAL30_SP50_GQ25.vcf.gz

bcftools index F5_refCol0_B1_2025_noindel_MQ40_QUAL30_SP50_GQ25.vcf.gz

NB_pool=$(bcftools query -l F5_refCol0_B1_2025_noindel_MQ40_QUAL30_SP50_GQ25.vcf.gz | wc -l)
echo "Number of pools after filtering the F2 and outliers is: $NB_pool"

NB_SNP=$(bcftools view -v snps F5_refCol0_B1_2025_noindel_MQ40_QUAL30_SP50_GQ25.vcf.gz | grep -v '^#' | wc -l)
echo "Number of SNP is: $NB_SNP"

```

```

bcftools query -f '%CHROM[\t%SAMPLE\t%GT]\n'
F5_refCol0_B1_2025_noindel_MQ40_QUAL30_SP50_GQ25.vcf.gz \
| awk '
{
    chr = $1
    for (i=2; i<=NF; i+=2) {
        s = $i
        gt = $(i+1)
        if (gt != "./." && gt != ".") {
            key = chr FS s
            cnt[key]++
        }
    }
}
END {
    for (k in cnt) print k, cnt[k]
}' OFS='\t' \
| sort -k1,1 -k2,2 > NB_SNP_F5_per_chr_per_sample.tsv

```

```

#####
# 2) KEEP SEGREGATING SNP IN THE F2 POOLS
#####
echo "=== Filtering the segregating SNP ==="

```

```

bcftools view \
-T sites_genic_DPclean_noMissingGT_AF0.4_0.6.txt \
-Oz -o F5_refCol0_B1_2025_noMissingGT_genic_DPclean.vcf.gz \
F5_refCol0_B1_2025_noindel_MQ40_QUAL30_SP50_GQ25.vcf.gz

```

```

bcftools index F5_refCol0_B1_2025_noMissingGT_genic_DPclean.vcf.gz

```

```

bcftools query -f '%CHROM[\t%SAMPLE\t%GT]\n' F5_refCol0_B1_2025_noMissingGT_genic_DPclean.vcf.gz \
| awk '
{
    chr = $1
    for (i=2; i<=NF; i+=2) {
        s = $i
        gt = $(i+1)
        if (gt != "./." && gt != ".") {
            key = chr FS s
            cnt[key]++
        }
    }
}
END {
    for (k in cnt) print k, cnt[k]
}' OFS='\t' \
| sort -k1,1 -k2,2 > NB_SNP_F5_per_chr_per_sample_SegregatingSNP.tsv

```

```

#####
# 3) ALLELIC FREQUENCY IN THE F5 POOLS
#####
echo "=== Extracting the allelic frequency of the F5 pools in R friendly table ==="

```

```

bcftools query \
-f '%CHROM\t%POS\t%REF\t%ALT[\t%SAMPLE\t%GT\t%AD\t%DP]\n' \
F5_refCol0_B1_2025_noMissingGT_genic_DPclean.vcf.gz \
| awk '
{
    chrom=$1; pos=$2; ref=$3; alt=$4;
    # fields from 5 onwards are groups of 4: SAMPLE, GT, AD, DP
    for (i=5; i<=NF; i+=4) {
        sample=$(i); gt=$(i+1); ad=$(i+2); dp=$(i+3);
    }
}

```

```

    altfreq="NA";
    if (ad != "." && dp != "." && dp > 0) {
      split(ad,a,","); # a[1]=REF reads, a[2]=ALT reads (for biallelic)
      altfreq = a[2] / dp;
    }
    print chrom"\t"pos"\t"ref"\t"alt"\t"sample"\t"gt"\t"ad"\t"dp"\t"altfreq;
  }
}
' > F5_refCol0_B1_2025_noMissingGT_genic_DPclean_RfriendlyAF.tsv
# Column header: CHROM POS REF ALT SAMPLE GT AD DP ALTFREQ

echo "=== May the force be with you ==="

```

IV. Allelic frequency in F5 pools

1. Adjust the ref genome for Mil2

In R modify the allelic frequency so the Mil2 allele is always the reference
when Mil2 = 1/1 & Kad1 = 0/0 I changed the allelic frequency for 1- Altfrq

```

module load lang/R/4.2.2-foss-2022b
R

library(readr)
library(dplyr)
library(tidyr)
library(data.table)
library(ggplot2)

# Importing F5 allelic frequency table
F5_frequency <- read_tsv("F5_refCol0_B1_2025_noMissingGT_genic_DPclean_RfriendlyAF.tsv")

F5_frequency <- rbind(colnames(F5_frequency), F5_frequency)
colnames(F5_frequency) <- c("Chrom", "Pos", "Ref", "Alt", "Sample_ID", "GT", "AD", "TotDP", "Altfreq")
F5_frequency$Chrom <- as.factor(F5_frequency$Chrom)
F5_frequency$Pos <- as.numeric(F5_frequency$Pos)
F5_frequency$Ref <- as.factor(F5_frequency$Ref)
F5_frequency$Alt <- as.factor(F5_frequency$Alt)
F5_frequency$Sample_ID <- as.factor(F5_frequency$Sample_ID)
F5_frequency$GT <- as.factor(F5_frequency$GT)
F5_frequency$Altfreq <- as.numeric(F5_frequency$Altfreq)

table(F5_frequency$GT)
      ./      0/0      0/1      1/1
529592 1006015 10428032  759521

F5_frequency <- F5_frequency %>% filter(GT != "./.")
summary(F5_frequency)

# Importing Parents genotypes
Mil2_Kad1_GT <- read_tsv("Mil2_Kad1_GT_positions.txt")

Mil2_Kad1_GT <- rbind(colnames(Mil2_Kad1_GT), Mil2_Kad1_GT)
colnames(Mil2_Kad1_GT) <-
c("Chrom", "Pos", "Ref", "Alt", "Mil2", "GT_Mil2", "AD_Mil2", "TotDP_Mil2", "Kad1",
  "GT_Kad1", "AD_Kad1", "TotDP_Kad1")
Mil2_Kad1_GT$Chrom <- as.factor(Mil2_Kad1_GT$Chrom)
Mil2_Kad1_GT$Pos <- as.numeric(Mil2_Kad1_GT$Pos)
Mil2_Kad1_GT$GT_Mil2 <- as.factor(Mil2_Kad1_GT$GT_Mil2)
Mil2_Kad1_GT$GT_Kad1 <- as.factor(Mil2_Kad1_GT$GT_Kad1)
summary(Mil2_Kad1_GT)

# Create a new column, we want Mil2 as Ref so we will adjust if

```

```

Mil2_Kad1_GT$Ref_GT <- "Ref_allele"
Mil2_Kad1_GT <- Mil2_Kad1_GT %>%
  mutate(Ref_GT = ifelse(GT_Mil2 == "1/1" & GT_Kad1 == "0/0", "Flip_ref", "No_change"))
Mil2_Kad1_GT <- Mil2_Kad1_GT %>%
  mutate(Ref_GT = ifelse(GT_Mil2 == "1/1" & GT_Kad1 == "0/1", "Flip_ref", Ref_GT))

#' Merge the two data set with only the position in the F5 population
F5_frequency <- left_join(F5_frequency, Mil2_Kad1_GT[,c(1,2,13)], by = c("Chrom", "Pos"))
F5_frequency$Ref_GT <- as.factor(F5_frequency$Ref_GT)

#' Modify the allelic frequency of the ALT with 1 - ALT at each position marked by Change_ALT
F5_frequency <- F5_frequency %>%
  mutate(Altfreq_modify = ifelse(Ref_GT == "Flip_ref", 1-Altfreq, Altfreq))

Chrom  Pos Ref  Alt  Sample_ID GT  AD  TotDP Altfreq Ref_GT  Altfreq_modify
<fct> <dbl> <fct> <fct> <fct> <fct> <chr> <dbl> <dbl> <fct> <dbl>
2 Chr1  6539 A    G    159    0/0  26,0  26  0    Flip_ref...  1
3 Chr1  6539 A    G    173    0/1  48,9  57  0.158 Flip_ref...  0.842
4 Chr1  8176 G    T    159    0/1  18,6  24  0.25  No_change  0.25
5 Chr1  8176 G    T    173    0/1  10,42  52  0.808 No_change  0.808
6 Chr1  8329 C    T    113    0/1  9,46  55  0.836 No_change  0.836

write_tsv(F5_frequency, file = "F5_Test_segregatingSNP_RfriendlyAF_refMil2.tsv")

```

2. Median allelic frequency on 50,000 bp windows

- Map AFs and keep all values per window, then Derive number of SNPs and number of NAs
Computes the median AF and also a comma-separated list of all AF values overlapping each window

median_frequency_per_50kb_windows.sh

```

#!/bin/bash
set -euo pipefail

#####
# Median allelic frequency on 50,000 bp windows
# Script from Justine Floret
# 01.2026
#####

#SBATCH --nodes=1
#SBATCH --time=24:00:00
#SBATCH --cpus-per-task=8
#SBATCH --mem=10gb
#SBATCH --account=ag-demeaux
#SBATCH --error=/projects/ag-demeaux/jfloret/Ath_Cologne_2025/SNP_calling/vcf_refCol0/F5_allelic_frq_refMil2/vcffiltering_%j.err
#SBATCH --output=/projects/ag-demeaux/jfloret/Ath_Cologne_2025/SNP_calling/vcf_refCol0/F5_allelic_frq_refMil2/vcffiltering_%J.out

module purge
module load bio/BEDTools/2.31.1-GCC-13.2.0

# set working directory
cd /projects/ag-demeaux/jfloret/Ath_Cologne_2025/SNP_calling/vcf_refCol0/F5_allelic_frq_refMil2

#####
# 1. Get chromosome lengths
#####
echo "=== Get chromosome lengths ==="

cut -f1,2 /projects/ag-demeaux/jfloret/Ath_Cologne_2025/Ref_genome/TAIR10_chr_all.fas.fai | grep -E '^Chr[1-5]' > chr_lengths.txt

```

```
#####
# 2. Create 50000 bp windows for each chromosome
#####
echo "=== Create 50000 bp windows for each chromosome ==="

bedtools makewindows -g chr_lengths.txt -w 50000 > windows_50kb.bed

#####
# 3. Create per sample Bed Allelic frequency table
#####
echo "=== Create per sample Bed Allelic frequency table ==="

while read SAMPLE; do
    tail -n +2 ../F5_refCol0_B1_2025_noMissingGT_genic_DPclean_RfriendlyAF_refMil2.tsv \
    | awk -v S="$SAMPLE" 'BEGIN{OFS="\t"} $5==S {print $1, $2-1, $2, $11}' \
    > "${SAMPLE}.AF.bed"
done < ../F5_list.txt

#####
# 4. Compute the median AF per window, per sample, per chromosome
#####
echo "=== Compute the median AF per window ==="

while read SAMPLE; do
    bedtools map \
    -a windows_50kb.bed \
    -b "${SAMPLE}.AF.bed" \
    -c 4,4 \
    -o median,collapse \
    > "${SAMPLE}.AF_50kb_median_raw.bed"

    awk 'BEGIN{FS=OFS="\t"}
    {
        af_list = $5
        if (af_list == ".") {print $1,$2,$3,$4,0,0; next}
        n_total = split(af_list, a, ",")
        n_na = 0
        for (i=1; i<=n_total; i++) if (a[i]=="NA" || a[i]==".") n_na++
        print $1, $2, $3, $4, n_total
    }' "${SAMPLE}.AF_50kb_median_raw.bed" > "${SAMPLE}.AF_50kb_median.bed"
done < ../F5_list.txt

#####
# 5. Merged in one table
#####
echo "=== Merged in one table ==="

while read SAMPLE; do
    awk -v S="$SAMPLE" 'BEGIN{OFS="\t"} {center = int(($2+$3)/2); print $1, center, S, $4, $5, $6}' \
    "${SAMPLE}.AF_50kb_median.bed"
done < ../F5_list.txt > F5_pools_50kb_medianAF.tsv

echo "=== May the force be with you ==="
```

``${SAMPLE}.AF_50kb_median.bed``

col1: chromosome

col2: window start (0-based)

col3: window end

col4: median AF

col5: number of SNP records in the window (`n_total`)

F5_pools_50kb_medianAF.tsv

col 1: Chrm,

col 2: Pos middle of the window

Col 3: Sample_ID

Col 4: median_AF

Col 5: SNP_tot